



BIRMINGHAM CITY
University



TraceOn

MAXIMIZE WORK, MINIMIZE SLACKING

**TraceOn: Tackling Remote Workforce Accountability with
Intelligent Activity-Based Screenshot Monitoring**
Minimum Viable Product

Anush Neupane

anush.neupane@mail.bcu.ac.uk

Bibek Karki

bibek.karki@mail.bcu.ac.uk

Nisan Gayak

nishan.gayak@mail.bcu.ac.uk

Utsav Rai

utsav.rai@mail.bcu.ac.uk

Faculty of Computing, Engineering and the Built Environment
Birmingham City University
Birmingham, United Kingdom

June 11, 2025

Abstract

TraceOn is an advanced monitoring system for activities that seek to improve accountability of remote workers while ensuring that privacy is not violated. The MVP (Minimum Viable Product) comes with a Python worker application that captures screenshots at user-defined intervals or when users are actively engaged. User-initiated-triggered screenshots are captured with OCR-based redaction filtering of sensitive information to ensure privacy. The managers can view the screenshots via a Flask dashboard which stores data securely on Google Drive. The core functionalities in place now include authentication and screenshot capturing and viewing. Future work on the system aims to incorporate AI classification and cross-platform functionality. Ethically-balanced productivity monitoring makes TraceOn a modern scalable solution to problems posed by remote work.

Keywords- *TraceOn, monitoring system, screenshot capturing, minimum viable product, privacy protection, OCR-based redaction, flask dashboard, google drive storage, authentication, productivity tracking*

INTRODUCTION

The shift to remote and hybrid workplaces has become an emblem of the contemporary workforce. Besides enabling flexibility and collaboration on a global scale, the shift has complicated the ability to measure productivity and enforce accountability on employees (Tran, 2022). According to the study (Vargo et al., 2021), supervision is no longer possible in virtual settings, and most existing digital monitoring solutions are either too blind to contextual considerations or breach user privacy in some way.

Organizations need a solution that strikes a balance between progress monitoring and respecting employee privacy and autonomy (Morris, 2025). It is common for employers to face challenges confirming if remote employees are genuinely interacting with work processes and not just logged into systems passively (TrackOLap, 2025). In the meantime, employees are more concerned than ever about trust-breaking invasive surveillance practices such as continuous screen recording or keystroke logging (Connolly, 2025).

This project, TraceOn, is developed in response to this critical dilemma. It aims to answer the following question:

- *How can employers monitor remote work activities in a non-intrusive, transparent, and context-aware manner that supports both productivity tracking and digital privacy?*

TraceOn presents a new hybrid technique which overlays periodic screenshot capturing with smart detection of changes such as new activity windows, tabs, or even domains. Furthermore, it employs OCR to scrub screenshots that may contain sensitive and private information like passwords or account details. The system attempts to solve the problem of

balancing monitoring and privacy in digital workplaces by storing information in a cloud structure and granting managerial web access.

Objectives: This report is designed to showcase the implementation, design, and evaluation of the TraceOn system while demonstrating how such a system can resolve issues of ethical productivity monitoring in remote work environments.

LITERATURE REVIEW

(Wieser and Abraham, 2024) examined the acceptance of various forms of digital surveillance, especially screen monitoring for remote employees. The study utilized surveys and regression analysis, finding that the degree of invasiveness has a direct impact on acceptance. In this case, acceptance of screen-based monitoring is stagnated regardless of how many coworkers are remote. This indicates that TraceOn might improve acceptance by reducing perceived intrusiveness with its OCR-based filtering and context-aware triggers.

Workers' reactions to digital surveillance were explored by (Sum et al., 2025) using interviews and qualitative analysis of discussions on Reddit. They found that monitored employees resort to countermeasures—from technical workarounds to mutual support—to resist surveillance, particularly when they are vulnerable to such an intrusion. Incorporating more transparency, control, and worker consent in the design of TraceOn could minimize such resistance and promote trust.

(Nurse et al., 2021) investigate the problems of cybersecurity and privacy simultaneously in relation to remote work and note that the haste in deploying surveillance technologies often leads to a breach of personal data security . Their recommendations suggest incorporating privacy features by design which is in harmony with the principle followed by TraceOn that employs OCR-based sensitive data redaction along with secure and cloud-based storage of information.”

(Jeske, 2022) provides specialized advice for health professionals and HR managers who work with remotely monitored employees. Their focus is on using monitoring data to actively support well-being by enforcing ergonomics and taking breaks instead of measuring productivity. It would be an improvement to TraceOn if it used data on screenshot frequency to suggest breaks which would enhance both productivity and employee wellness.

METHODOLOGY

The Minimum Viable Product (MVP) of TraceOn system was designed with a two-layer structure to achieve remote employee monitoring, privacy safeguarding, and centralized control. It consists of:

- A Python desktop application (with a GUI through Tkinter) for employee registration and login, as well as real-time activity tracking.
- Oversight web dashboards allowing user management and authentication with activity overview for assigned workers via cloud-stored screenshots built in Flask.

Software Components Used in TraceOn

Client-Side (Worker Application)

- **Python** – Core programming language for application logic.
- **Tkinter** – GUI library for creating the desktop interface (login, monitoring controls).
- **PyAutoGUI** – For capturing screenshots of the user's screen.
- **PyGetWindow** – For detecting active windows/tabs/domains.
- **Psutil** – To interact with system processes and monitor application-level activity.
- **Pytesseract (Tesseract OCR)** – To extract text from screenshots and filter sensitive content.
- **Requests** – For sending HTTP requests to the server APIs (e.g., overseer validation, worker registration).
- **SQLite** – Lightweight local database to store user credentials and records.
- **Threading** – To handle background screenshot capture without freezing the GUI.
- **Drive Utils Module (Custom)** – For Google Drive integration (folder creation and file upload).

Server-Side (Manager Dashboard)

- **Python (Flask)** – Lightweight web framework to build the dashboard and REST APIs.
- **Flask-CORS** – Enables secure cross-origin communication with the client application.
- **Werkzeug Security** – For hashing and verifying user passwords securely.
- **Jinja2 (via Flask)** – For rendering dynamic HTML templates.
- **SQLite** – Used to store overseer and worker details (users.db, manager.db).
- **Google Drive API (via drive_utils)** – To manage and share folders and screenshots securely on the cloud.

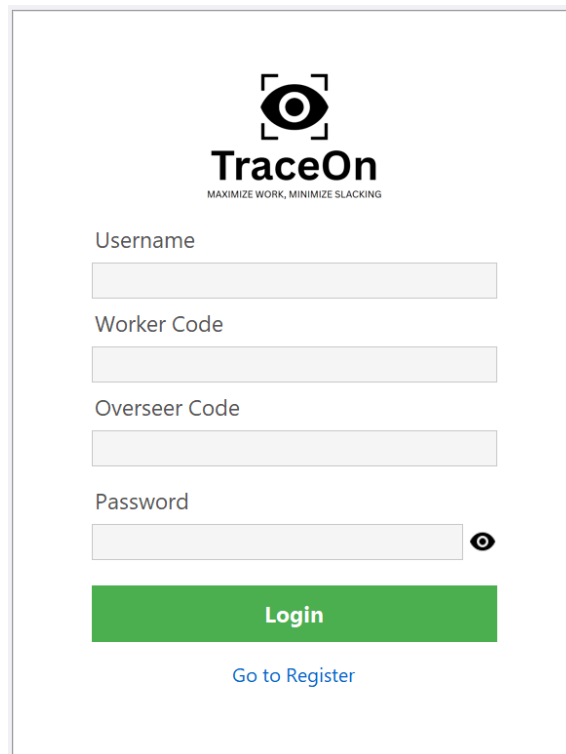
Worker Application

The cloud-shared screenshot system executes task management and encrypted login, user validation, role confirmation, activity monitoring, time tracking, screenshot documentation, cloud uploading, and synchronization. Its major components are:

- **Authentication & Registration**

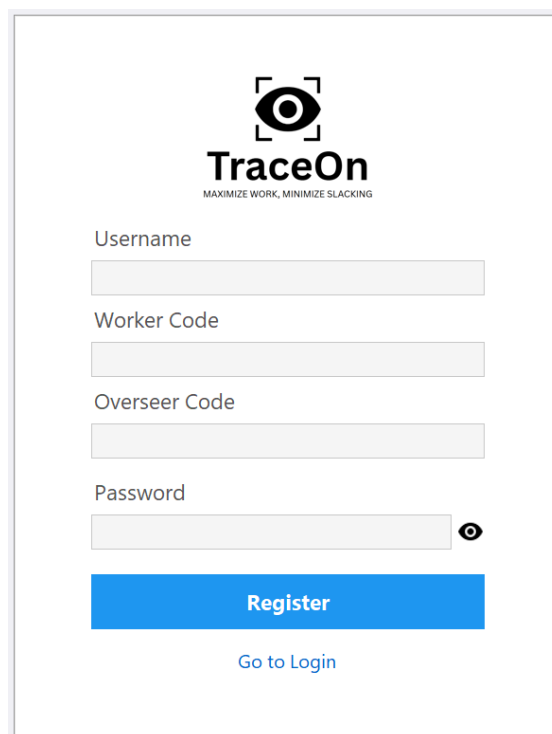
A worker's username, worker code, overseer code, and password can all be entered into one interface. The password and the provided credentials are checked against a

local SQLite database after being hashed. The overseer code is checked using an API call to the Flask server during registration as well, to ensure successful registration of the worker only if the manager with the entered overseer code already exists in the database of the web-based manager dashboard.



The image shows the login portal for the TraceOn Worker Application. At the top, there is a logo consisting of a stylized eye with a camera-like frame, followed by the text "TraceOn" and the tagline "MAXIMIZE WORK, MINIMIZE SLACKING". Below the logo, there are four input fields: "Username", "Worker Code", "Overseer Code", and "Password". The "Password" field has a small eye icon to its right, indicating a toggle for visibility. Below the input fields is a green "Login" button. At the bottom, there is a blue link that says "Go to Register".

Fig.-1: Login portal of Worker Application



The image shows the registration portal for the TraceOn Worker Application. It features the same logo and tagline as the login portal. Below the logo, there are four input fields: "Username", "Worker Code", "Overseer Code", and "Password". The "Password" field has a small eye icon to its right. Below the input fields is a blue "Register" button. At the bottom, there is a blue link that says "Go to Login".

Fig.-2: Registration portal of Worker Application

- **Interface after successful authentication**

After successful authentication, a simple and easy to use interface opens up consisting of three components,

1. **Start Button:** Pressing the start button initializes the screenshot process.
2. **Stop Button:** Pressing the stop button stops the screenshot process.
3. **Status:** Status indicator indicates whether the system is “on” or “off”.

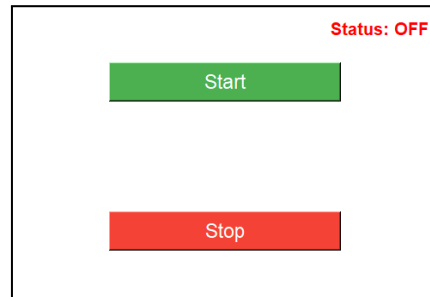


Fig.-3: Interface after Successful Login

- **Activity Monitoring Logic**

This application does not cease operating after a user logs in. It spawns a background thread for the following:

- Take a screenshot every 15 minutes.
- Monitors for new window/tab/domain activity with pygetwindow and psutil. A screenshot is captured instantly when a new window title is detected.

```
def monitor_and_screenshot(self, uname, wcode, ocode):
    self.monitoring = True
    seen_windows = set()
    last_periodic_time = time.time()

    while self.monitoring:
        try:
            now = time.time()

            # --- Detect new windows ---
            new_window_titles = set()
            for proc in psutil.process_iter(['pid', 'name']):
                try:
                    windows = gw.getWindowsWithTitle(proc.info['name'])
                    for win in windows:
                        if win.title.strip() != "":
                            new_window_titles.add(win.title)
                except:
                    continue

            new_titles = new_window_titles - seen_windows
            if new_titles:
                for title in new_titles:
                    print(f"[New] Detected: {title}")
                    self.capture_and_upload_screenshot(uname, wcode, ocode)
                seen_windows.update(new_titles)

            # --- Take periodic screenshot every 15 minutes ---
            if now - last_periodic_time >= 15 * 60:
                print("[Timer] Taking periodic screenshot...")
                self.capture_and_upload_screenshot(uname, wcode, ocode)
                last_periodic_time = now

        except Exception as e:
            print(f"[Error] {e}")

        time.sleep(2)
```

Fig.-4: Code snippet of Activity Monitoring Logic

- **Privacy-Aware Filtering**

To protect user privacy, screenshots undergo OCR processing via Tesseract to detect sensitive keywords such as passwords and PINs. If any matches are found, the screenshot is discarded.

```
def is_private_content(self, image):  
    """Check if screenshot contains private info using OCR and keyword filter."""  
    try:  
        text = pytesseract.image_to_string(image).lower()  
        sensitive_keywords = ["password", "otp", "pin", "username", "gmail", "inbox", "account  
number"]  
        return any(word in text for word in sensitive_keywords)  
    except Exception as e:  
        print(f"[OCR Error] {e}")  
        return False
```

Fig.-5: Code snippet of Privacy-Aware Filtering

- **Folder Management & Cloud Integration (Google Drive)**

The system makes use of a defined folder hierarchy system to capture and store screenshots in an orderly manner. At the topmost level, there is a root folder **“TraceOnScreenshots”** in the main cloud account. Inside this root folder, there are subordinate folders for each overseer which makes accessing data for each worker convenient via the manager dashboard which is web based. Inside each overseer folder, there are additional subfolders which use a mixture of the worker's name and their worker code. These subfolders contain folders for daily screenshots which are dated (e.g., “2025-06-11”). Within these date-specific folders, all screenshots taken on that day are stored. This entire process of folder creation is done automatically which removes the need of manual intervention and increases the efficiency of data organization. The folder is automatically populated with the screenshots via Google Drive APIs.

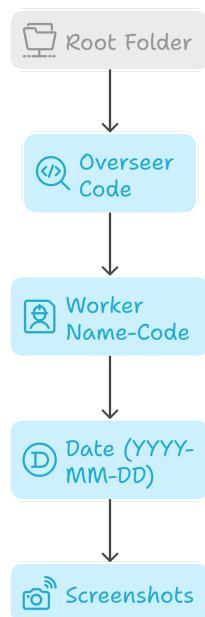


Fig.-6: Hierarchy of folders followed by TraceOn

Web-Based Manager Dashboard

The manager dashboard is a web application built using Flask that allows overseers to register, log in, and access the screenshot history of their workers. The application controls role-based secure access to organized folder level screenshot records stored on Google Drive while preserving authentication, data integrity, and overall usability.

- **User Authentication and Access Control**
 - **Registration:** Overseers registration includes entering username, email, a unique 4 digit overseer code and a strong password which is checked for complexity. The system prevents duplicates, checks email formats, and hashes passwords via werkzeug.security.

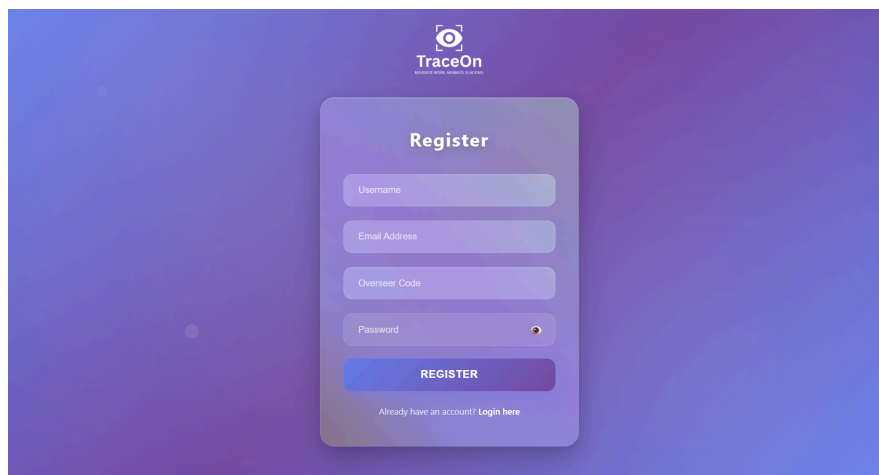
The image shows the registration page of the TraceOn web application. The page has a purple and blue gradient background. At the top center is the TraceOn logo, which consists of a stylized eye icon and the text "TraceOn" with the tagline "MANAGING YOUR WORKERS' SCREENS" below it. Below the logo is a white rounded rectangle containing the registration form. The form is titled "Register" in bold. It contains four input fields: "Username", "Email Address", "Overseer Code", and "Password". The "Password" field has a small eye icon to its right. Below the input fields is a blue button labeled "REGISTER". At the bottom of the form, there is a link that says "Already have an account? Login here".

Fig.-7: Registration page of Web-Based Manager Dashboard

- **Login:** Login success allows creation of session variables which store the overseer's code and credentials. These are used to filter and control access to pertinent workers and data.

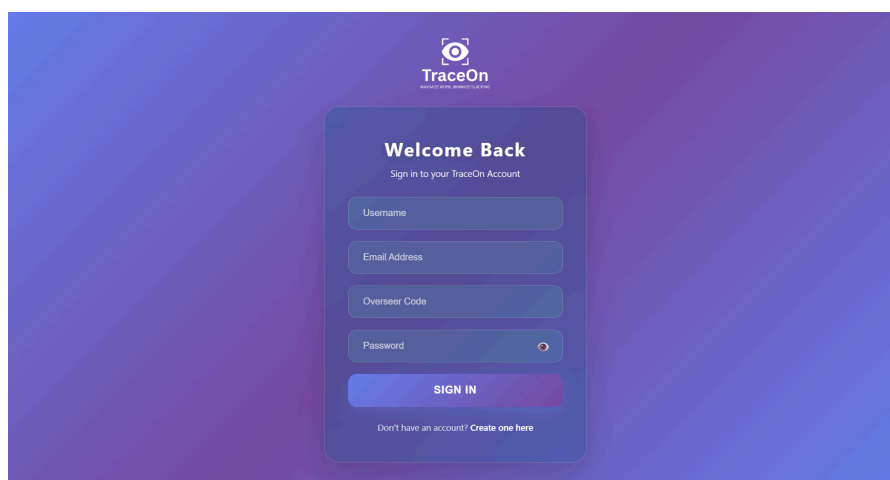
The image shows the login page of the TraceOn web application. The page has a purple and blue gradient background. At the top center is the TraceOn logo, which consists of a stylized eye icon and the text "TraceOn" with the tagline "MANAGING YOUR WORKERS' SCREENS" below it. Below the logo is a white rounded rectangle containing the login form. The form is titled "Welcome Back" in bold, with the subtitle "Sign in to your TraceOn Account" below it. It contains four input fields: "Username", "Email Address", "Overseer Code", and "Password". The "Password" field has a small eye icon to its right. Below the input fields is a blue button labeled "SIGN IN". At the bottom of the form, there is a link that says "Don't have an account? Create one here".

Fig.-8: Login page of Web-Based Manager Dashboard

- **Session Management:** Active user sessions are managed with Flask's session object. Users without valid sessions are redirected to log in.
- **Worker Synchronization**
 - The dashboard has an API feature that gives an endpoint for the worker-side application to register new workers seamlessly (/api/register_worker).
 - All registered workers are retained in a distinct “manager.db” database alongside their corresponding username, worker_code, and overseer_code.
 - The system is designed to prevent duplicate entries and allows multiple workers to be registered under one overseer.

```

@app.route('/api/register_worker', methods=['POST'])
def register_worker():
    data = request.get_json()

    required_fields = ['username', 'worker_code', 'overseer_code']
    if not all(field in data for field in required_fields):
        return jsonify({"success": False, "error": "Missing fields"}), 400

    username = data['username']
    worker_code = data['worker_code']
    overseer_code = data['overseer_code']

    # Connect to the manager website's DB
    conn = sqlite3.connect('manager.db')
    cursor = conn.cursor()

    # Optional: Create table if not exists
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS workers (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            username TEXT,
            worker_code TEXT,
            overseer_code TEXT
        )
    ''')

    # Prevent duplicates
    cursor.execute("SELECT * FROM workers WHERE worker_code=? AND overseer_code=?", (worker_code,
    overseer_code))
    if cursor.fetchone():
        conn.close()
        return jsonify({"success": False, "error": "Worker already exists"}), 409

    # Insert worker
    cursor.execute("INSERT INTO workers (username, worker_code, overseer_code) VALUES (?, ?, ?)",
    (username, worker_code, overseer_code))
    conn.commit()
    conn.close()

    return jsonify({"success": True}), 201

```

Fig.-9: Code snippet of Worker Synchronization

- **Viewing Screenshots**

After completing login, the manager's role dashboard is updated to show the manager dashboard page. On the manager dashboard page, the only actionable elements available for the MVP (minimum viable product) are “View Screenshots” and “Logout.” Other elements like statistics for active workers, screenshots, uptime metrics, along with several navigation links labeled Dashboard, Reports, Contact Admin, are nonfunctional placeholders and will be augmented in the forthcoming iterations which aim to enrich platform capabilities. This strategy guarantees that essential features are focused on first and then iterative enhancements can be made systematically.

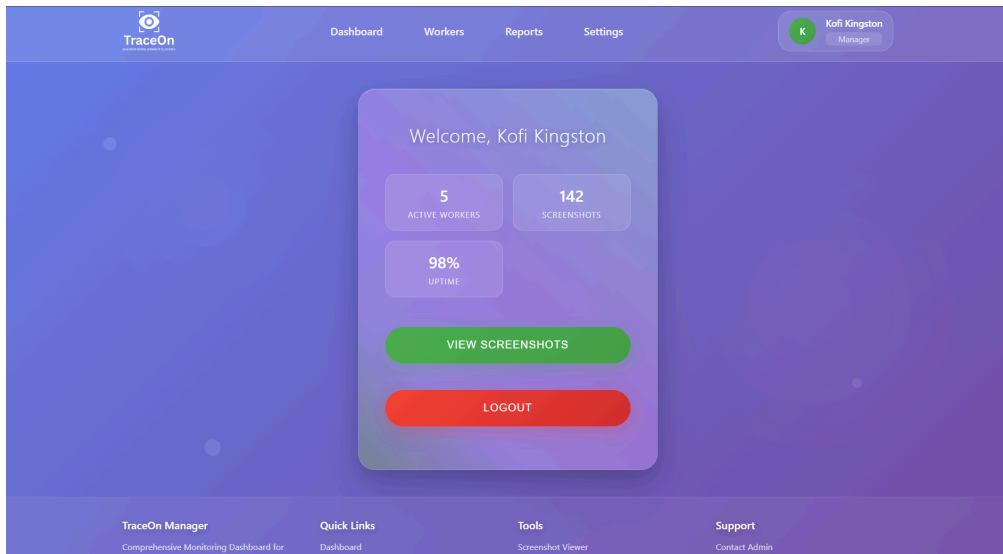


Fig.-10: Dashboard page of Web-Based Manager Dashboard

Clicking on the “View Screenshots” button leads to a page where the registered workers of the manager are listed (i.e. workers who had entered the overseer code of the manager during registration from the worker application).

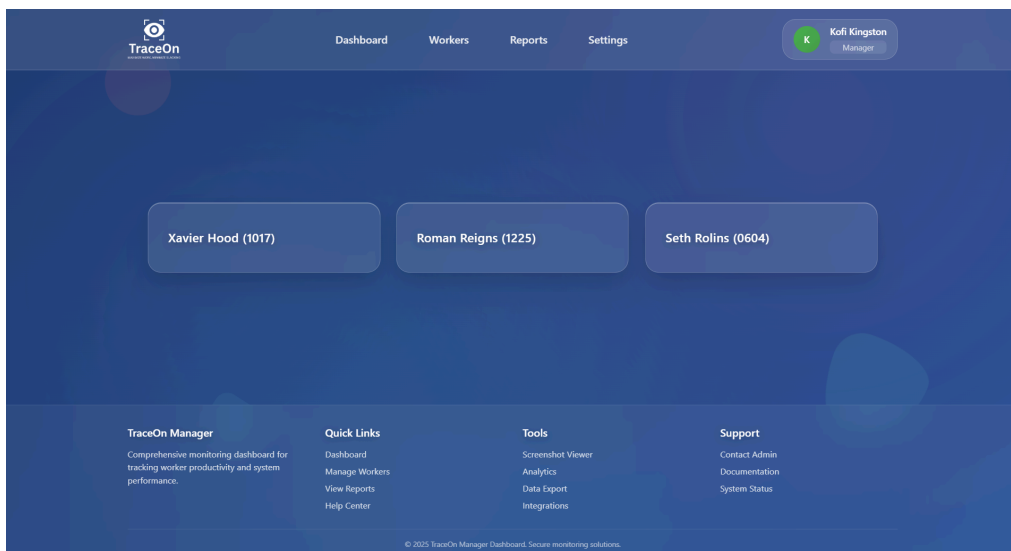


Fig.-11: Workers page of Web-Based Manager Dashboard

Clicking on a worker’s button will further lead to the dated folder page which will contain folders named after the date of the screenshots that the folder’s storing inside it.

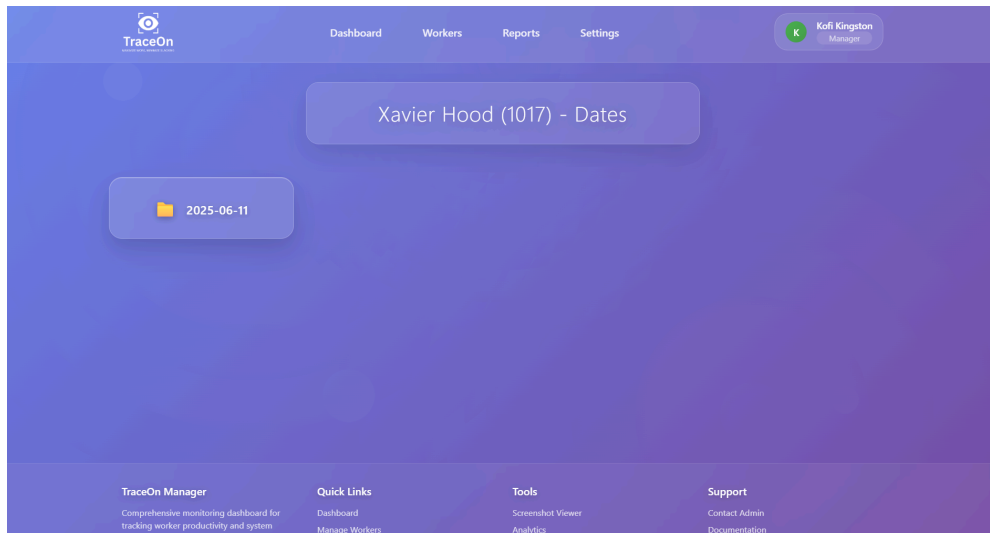


Fig.-12: Dated folder page of Web-Based Manager Dashboard

Clicking on a folder navigates the user to a screenshot page that displays all screenshots captured on the specific day represented by the folder's name.

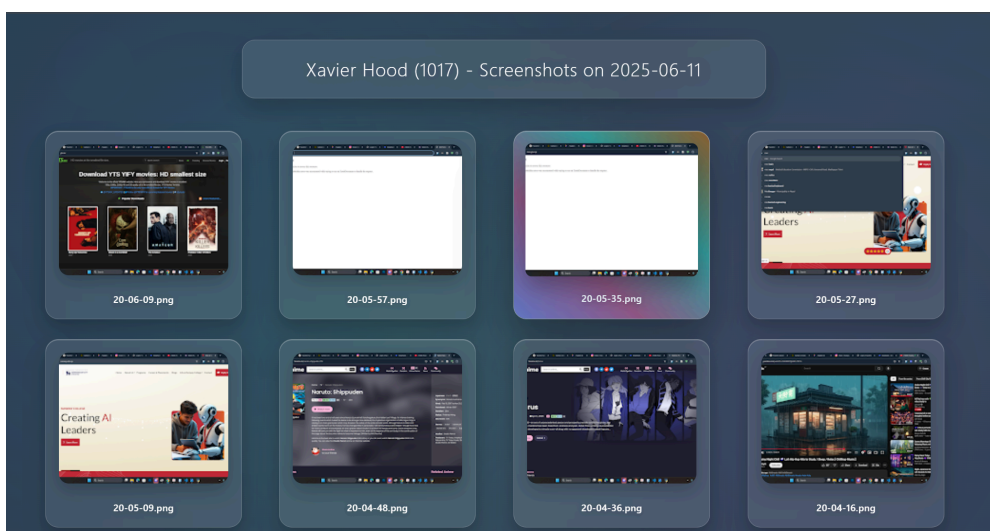


Fig.-13: Screenshot page of Web-Based Manager Dashboard

The Google Drive account verification system restricts access to full-size screenshots to authorized overseers only. When an overseer signs up on the system, a folder with the name of their overseer code is created inside a main folder called “TraceOnScreenshots” which houses all overseer folders. That folder is then automatically shared with the Google Drive account linked to the email given during registration. Hence, only when logged into Google Drive with the registered email, will the overseer be able to unlock and view the full-size screenshots by clicking the images on the screenshot page.

This system makes sure that the sensitive visual data is only accessible by the right person enhancing the security and privacy of the system. Unauthorized users, even

when having access to the dashboard interface, will not be able to unlock complete screenshots without prior Google Drive account authentication through the registration provided during signup.

Workflow of Worker Application

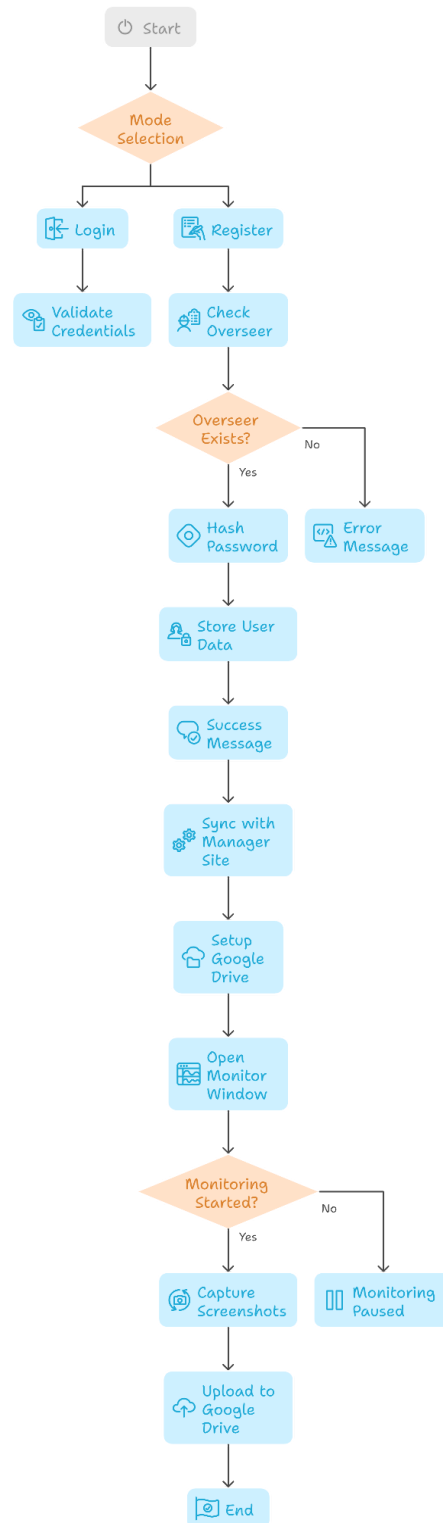


Fig.-14: Workflow of Worker Application

Workflow of Web-based Manager Dashboard

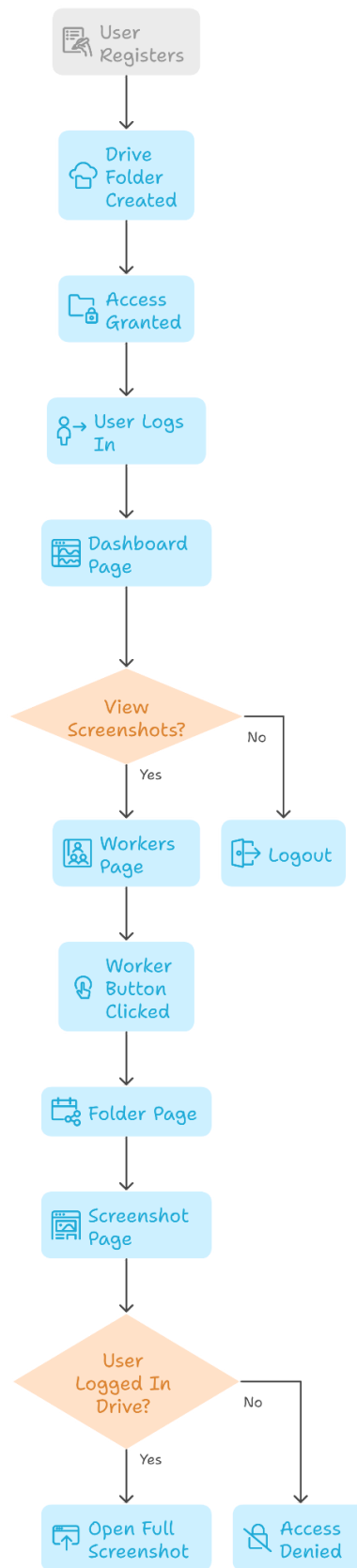


Fig.-15: Workflow of Web-Based Manager Dashboard

TraceOn Client-Server Architecture Comparison

Characteristic	Client (Worker App)	Server (Manager Dashboard)
 Key Roles	Collects credentials, captures screenshots, filters content, uploads to Drive	Overseer registration, validates codes, stores info, displays screenshots, controls Drive access
 Functional Flow	UI, Overseer verification, worker registration, monitoring, screenshot upload	Overseer registration/login, API endpoints, screenshot viewing, authorization
 API Interactions	HTTP APIs, Google Drive APIs	<code>`/register`</code> , <code>`/login`</code> , <code>`/api/check_overseer`</code> , <code>`/api/register_worker`</code>
 Data Storage	Uploads screenshots to Google Drive	Stores credentials in <code>`users.db`</code>
 Security	Filters private content using OCR	Session-based access, Drive folder sharing

Fig.-16: Comparison of Client-Server Architecture of TraceOn

Importance of MVP

Because of following advantages MVP deems important:

- **Validates the core idea early**
Evaluates if the concept solves a problem in reality before heavily committing resources of time and money.
- **Reduces development costs**
Avoids unnecessary expenditure by concentrating on crucial elements and essential features first.
- **Speeds up time-to-market**
Opens the possibility for earlier deployments so that stakeholders can gain access to results and feedback sooner.
- **Facilitates user feedback and iteration**
Users of the product can test it, as well as contribute ideas on how to improve it, providing useful insights for the product enhancement.
- **Minimizes risk of failure**
Reduces the chances of building unwanted products by validating assumptions early on testing critical features.
- **Aids in securing stakeholder or investor support**
Trust and opportunities for funding increase as the prototype shows progress, and potential adds to demonstrated MVP.
- **Supports rapid prototyping and experimentation**
Sparks continuous enhancements according to the actual utilization of the product and received feedback.
- **Clarifies the product vision**
Forces developers to prognose what the most impactful features are, thus, clarifying the vision of the product.

DISCUSSION

Challenges & Solutions

While working on TraceOn, there were some design and technical issues that arose. Each one was tackled using some form of planning and testing. Here, the most important issues to tackle in the development cycle of TraceOn is presented below:

Challenge	Solution
 Privacy vs. Monitoring	OCR filtering
 Real-time Activity Detection	Window title detection
 OCR Accuracy	Keyword focus and resolution
 Scalable Folder Structure	Unique codes and Drive API
 UI Responsiveness	Daemon thread
 Secure Communication	JSON-based HTTP POST requests
 Access Restriction	Email-based folder sharing
 Consistent User Experience	Clean GUI and responsive templates
 Data Duplication	Server-side checks

Fig.-17: Tabular Summary of Challenges & Solution Methods Applied

- Privacy vs. Monitoring**
 Employed OCR technology to mask sensitive information such as passwords and One Time Passwords (OTPs) in screenshots.
- Detecting Activity Changes**
 Utilized pygetwindow and psutil to capture screenshots after new windows or tabs appear.
- OCR Accuracy**
 Concentrated on detection accuracy related to prompts and optimized the filtering resolution for better reliability.
- Folder Management in Google Drive**
 Automated creation of hierarchical folders using overseer and worker IDs to prevent duplicates.

- **UI Responsiveness During Monitoring**
Real-time screenshot capturing was facilitated with GUI threading, which maintained responsiveness.
- **Secure Client-Server Communication**
Implemented secure JSON POST requests and CORS for API functions to enable secure communication.
- **Access Control for Managers**
Permitted shared folder access only to registered manager emails, restricting access through Google Drive login.
- **Consistent User Experience**
Developed interfaces with uniform design using Tkinter for workers and Flask templates for managers.
- **Duplicate Entry Prevention**
Implemented logic to restrict the registration of multiple overseers or workers.

Future Enhancements

- **Classification of Screenshots Using AI**
Develop an AI model that classifies screenshots into productive and non-productive categories with minimum human intervention.
- **Monitoring Settings Customization**
Allow higher management to define role-based conditions and delimiters for certain functions to determine the assignment of tasks and limits for screenshot capture.
- **Cloud Storage with Encryption**
Apply encryption to secure the images during the upload and storage phase which enhances data privacy and confidentiality.
- **Compatibility with Other Platforms**
Add support for MacOS and Linux to expand the system's accessibility to operating systems.
- **Instant Notification Alerts**
Create systems that alert the manager immediately after identification of some form of activity that is likely to be unproductive.

CONCLUSION

The challenges posed by managing remote workforces have necessitated the careful consideration of solutions like TraceOn. Achieving an intelligent balance between supervision and privacy, the system employs an activity-based monitoring strategy with intelligent oversight. Effective productivity monitoring coupled with strict boundary control

is made possible through the system's Google Drive integration with the worker and manager desktop and web interface applications.

Offering event-triggered and periodic screenshot capture alongside OCR privacy filtering and automated cloud folder organization, TraceOn remote monitoring ensures security and privacy while remaining intuitive and scalable. Addressing the core challenge of data accountability and access control, the project also provides a foundation for advanced future features such as AI-driven screenshot classification or performance analytics integration.

As demonstrated by the Minimum Viable Product, technology can promote accountability while fostering trust in distributed work environments. With further enhancements, TraceOn has the ability to evolve into an all-encompassing solution for contemporary workforce management.

REFERENCES

- Connolly, A. (2025) Employee Monitoring Statistics 2025: Trends, Tools, and Privacy Insights. *Hubstaff Blog*. Available at: <https://hubstaff.com/blog/employee-monitoring-statistics/> (Accessed: 11 June 2025).
- Jeske, D. (2022) Remote workers' experiences with electronic monitoring during Covid-19: implications and recommendations. *International Journal of Workplace Health Management*, 15 (3): 393–409. doi:10.1108/IJWHM-02-2021-0042.
- Morris, A. (2025) *Employee Monitoring: Benefits, Risks & Best Practices* | DavidsonMorris. Available at: <https://www.davidsonmorris.com/employee-monitoring/> (Accessed: 11 June 2025).
- Nurse, J.R.C., Williams, N., Collins, E., et al. (2021) *Remote Working Pre- and Post-COVID-19: An Analysis of New Threats and Risks to Security and Privacy*. In pp. 583–590. doi:10.1007/978-3-030-78645-8_74.
- Sum, C.M., Shi, C. and Fox, S.E. (2025) “It’s Always a Losing Game”: How Workers Understand and Resist Surveillance Technologies on the Job. doi:10.48550/arXiv.2412.06945.
- TrackOLap (2025) *What are the challenges of remote work time tracking?* Available at: <https://trackolap.com/blog/what-are-the-challenges-of-remote-work-time-tracking> (Accessed: 11 June 2025).
- Tran, L. (2022) *The Impact of Hybrid Work on Productivity: Understanding the Future of Work*.
- Vargo, D., Zhu, L., Benwell, B., et al. (2021) Digital technology use during COVID -19 pandemic: A rapid review. *Human Behavior and Emerging Technologies*, 3 (1): 13–24. doi:10.1002/hbe2.242.
- Wieser, L. and Abraham, M. (2024) Employee acceptance of digital monitoring systems

while working from home. *New Technology, Work and Employment*, 39 (1): 109–142.
doi:10.1111/ntwe.12280.